Technical Appendix to FasterBERT (Submission 4857)

Anonymous submission

Appendix

Introductions to more multi-exit BERT training methods

Two-stage training method The two-stage (2ST) (Xin et al. 2020; Liu et al. 2020) training strategy divides the training procedure into two stages. The first stage is identical to the vanilla BERT fine-tuning, updating the backbone model and only the final exit. In the second stage, we freeze all parameters updated in the first stage and fine-tune the remaining exits separately:

$$\text{Stage1} : \mathcal{L}_{stage1} = \mathcal{L}_M^{CE}(y_i, f_M(x_i; \theta_M)) \tag{1}$$

$$\text{Stage2} : \mathcal{L}_{stage2} = \mathcal{L}_m^{CE}, m = 1, ..., M-1. \tag{2}$$

where $\mathcal{L}_m^{CE} = \mathcal{L}_m^{CE}(y_i, f_m(x_i; \theta_m))$ denotes the cross-entropy loss of m-th exit.

Alternating training This method alternates between joint training and two-stage training across different optimization steps, and it was proposed by BERxiT (Xin et al. 2021):

$$\text{Odd} : \mathcal{L}_{stage1} = \mathcal{L}_M^{CE}(y_i, f_M(x_i; \theta_M)) \tag{3}$$

$$\text{Even} : \mathcal{L}_{joint} = \sum_{m=1}^{M} \mathcal{L}_m^{CE} \tag{4}$$

Statistics of the GLUE benchmark

Here we presented the in table 1 the details of the datasets on seven GLUE tasks.

Dataset	Labels	Train/Dev/Test
COLA	2	8.5k/1.0k/1.0k
RTE	2	2.5k/0.3k/3.0k
MRPC	2	3.7k/0.4k/1.7k
SST-2	2	67k/0.9k/1.8k
QNLI	2	105k/5.5k/5.5k
MNLI	3	393k/9.8k/9.8k
QQP	2	364k/40k/391k

Table 1: The statistics of the GLUE datasets.

Hyper-parameters for our FasterBERT

Table 2 presents the detailed hyper-parameters for the best performing FasterBERT model. The hyper-parameters we present are: (a) training batch size (bsz); (b) learning rate (lr); (c) activation functions g_1 and g_2 in the bypasses. From Table 2, most of the tasks favor the Tanh activation function as g_1, and reduce g_2 as the identity mapping.

	batch size	lr	g_1	g_2
RTE	16	1e-5	Tanh	Identity
CoLA	16	3e-5	Tanh	Identity
MRPC	16	2e-5	Tanh	Identity
SST-2	128	1e-5	GELU	Identity
QNLI	128	2e-5	Tanh	Identity
QQP	1024	3e-5	Tanh	Identity
MNLI	1024	5e-5	Tanh	Identity

Table 2: Hyper-prameters adopted by our best FasterBERT models on each GLUE tasks.

Bar plot for the first prediction-changing layers

In Figure 1, we plot the layer indices where the multi-exit ALBERT trained with our FasterBERT method change its prediction for the first time, on the RTE task. We can see that there are samples that prediction changes occur at layer 4 to 8.

Layer-score curves for four GLUE tasks

In the main content, we have plotted the layer-score curves for the CoLA, MRPC and SST-2 tasks. Here we present the layer-score curves for the RTE, QNLI, QQP and MNLI tasks (in Figure 2). The curves shown in Figure 2 demonstrate that our FasterBERT method outperforms the baseline models on most of the ALBERT layers. Especially, our method outperforms Single-Exits in just one model backbone, showing that our method is applicable in industry.

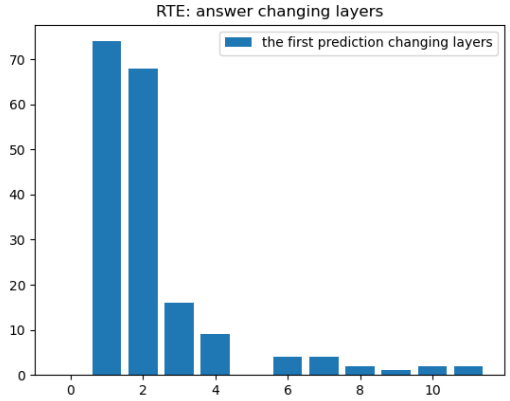


Figure 1: The layer indices where the multi-exit ALBERT change its prediction for the first time, on the RTE task.

Speedup-score curves for four GLUE tasks

In the main content, we have plotted the speedup-score curves for the CoLA, MRPC and SST-2 tasks. Here we present the speedup-score curves for the RTE, QNLI, QQP and MNLI tasks (in Figure 2). The curves shown in Figure 2 demonstrate that our FasterBERT method outperforms the baseline models on most of the ALBERT layers. Especially, our method outperforms Single-Exits in just one model backbone, showing that our method is applicable in industry.

Early exiting hyper-parameters for each method and each tasks.

Patience parameters for the Patience early exiting method Under this early exiting method, the patience parameters are {1,2,3,4,5,6,7,8,9,10,11} for all the GLUE tasks we run.

Thresholds for the Entropy early exiting method Since we want the points in the speedup-performance curves to be distributed evenly in the interval $(0, 1)$, we adjust the threshold parameter fo the entropy-based confidence scores on each task:

- On the RTE task: {0.99996, 0.99992, 0.99988, 0.9998, 0.99972, 0.99956, 0.99928, 0.9988, 0.9976, 0.9958, 0.9922, 0.9868, 0.98, 0.97, 0.96, 0.95, 0.93, 0.92, 0.9, 0.89, 0.875, 0.87, 0.85}.
- On the CoLA task: {0.968, 0.966, 0.965, 0.964, 0.963, 0.961, 0.96, 0.957, 0.95, 0.935, 0.92, 0.905, 0.885, 0.875, 0.87, 0.86, 0.855, 0.852, 0.85, 0.847, 0.846, 0.845, 0.844, 0.75}.
- On the MRPC task: {0.968, 0.966, 0.965, 0.964, 0.963, 0.961, 0.96, 0.957, 0.95, 0.935, 0.92, 0.905, 0.885, 0.875, 0.87, 0.86, 0.855, 0.852, 0.85, 0.847, 0.846, 0.845, 0.844, 0.75}.
- On the SST-2 task: {0.9868, 0.97, 0.959, 0.935, 0.915, 0.9, 0.89, 0.88, 0.875, 0.857, 0.855, 0.852, 0.848, 0.846, 0.845, 0.844, 0.843, 0.8418, 0.8414, 0.8412, 0.841, 0.84}.
- On the QNLI task: {0.99952, 0.9982, 0.9964, 0.9934, 0.9898, 0.985, 0.98, 0.968, 0.955, 0.94, 0.93, 0.915, 0.905, 0.895, 0.885, 0.88, 0.87, 0.86, 0.858, 0.852, 0.848, 0.845, 0.843, 0.84}.
- On the QQP task: {0.9964, 0.9892, 0.983, 0.975, 0.962, 0.955, 0.945, 0.93, 0.92, 0.91, 0.9, 0.885, 0.88, 0.87, 0.857, 0.853, 0.846, 0.844, 0.8418, 0.8412, 0.84}.
- On the MNLI task: {0.9982, 0.997, 0.9952, 0.9934, 0.991, 0.9873999999999999, 0.985, 0.98, 0.975, 0.967, 0.962, 0.955, 0.95, 0.945, 0.935, 0.93, 0.92, 0.915, 0.91, 0.905, 0.9, 0.895, 0.84}.

Thresholds and patience parameters for our DIVE method We adjust the divergence threshold parameter and the patience parameter on each task. The detailed (threshold, patience) pairs for each task are:

- On the RTE task: {(1, 0.695), (1, 0.693), (1, 0.692), (2, 0.697), (2, 0.695), (2, 0.693), (1, 0.6), (1, 0.51), (1, 0.44), (1, 0.4), (1, 0.34), (1, 0.29), (1, 0.26), (1, 0.23), (2, 0.29), (4, 0.6), (4, 0.45), (4, 0.37), (1, 0.15), (1, 0.14), (1, 0.13), (1, 0.12), (1, 0.11), (1, 0.1), (2, 0.1)}.
- On the CoLA task: {(1, 0.67), (1, 0.59), (1, 0.51), (1, 0.45), (1, 0.34), (2, 0.54), (2, 0.48), (2, 0.44), (3, 0.56), (2, 0.21), (3, 0.38), (3, 0.33), (4, 0.54), (1, 0.05), (1, 0.045), (2, 0.055), (2, 0.05), (1, 0.04), (3, 0.05), (3, 0.045), (2, 0.04), (1, 0.035), (5, 0.045), (1, 0.03), (1, 0.0)}.
- On the MRPC task: {(1, 0.685), (1, 0.63), (1, 0.59), (1, 0.52), (1, 0.47), (1, 0.41), (1, 0.32), (1, 0.28), (2, 0.39), (2, 0.38), (2, 0.23), (2, 0.2), (1, 0.1), (5, 0.63), (6, 0.715), (6, 0.706), (7, 0.739), (7, 0.67), (1, 0.05), (8, 0.67), (1, 0.046), (1, 0.045), (1, 0.043), (1, 0.041), (1, 0.02)}.
- On the SST-2 task: {(1, 0.62), (1, 0.46), (1, 0.34), (1, 0.28), (1, 0.2), (2, 0.43), (1, 0.13), (1, 0.1), (1, 0.085), (1, 0.06), (1, 0.049), (5, 0.695), (5, 0.56), (1, 0.03), (6, 0.685), (1, 0.02), (7, 0.685), (3, 0.03), (8, 0.751), (8, 0.685), (3, 0.02), (9, 0.754), (5, 0.02), (6, 0.02), (8, 0.02)}.
- On the QNLI task: {(1, 0.692), (1, 0.66), (1, 0.61), (1, 0.54), (1, 0.48), (1, 0.41), (1, 0.34), (1, 0.29), (1, 0.24), (1, 0.21), (1, 0.18), (1, 0.15), (1, 0.14), (1, 0.11), (1, 0.1), (1, 0.075), (1, 0.06), (1, 0.048), (1, 0.04), (1, 0.032), (1, 0.025), (1, 0.02), (1, 0.016), (1, 0.012), (1, 0.002)}.
- On the QQP task: {(1, 0.66), (1, 0.58), (1, 0.5), (1, 0.43), (1, 0.37), (1, 0.31), (1, 0.26), (1, 0.24), (1, 0.19), (1, 0.17), (2, 0.25), (2, 0.19), (2, 0.16), (1, 0.06), (1, 0.045), (1, 0.032), (1, 0.02), (1, 0.012), (1, 0.01), (1, 0.006), (1, 0.004), (1, 0.002)}.
- On the MNLI task: {(1, 1.04), (1, 0.97), (1, 0.9), (1, 0.86), (1, 0.78), (1, 0.709), (1, 0.65), (1, 0.58), (1, 0.52), (1, 0.46), (1, 0.4), (1, 0.35), (1, 0.3), (1, 0.29), (3, 0.5), (3, 0.39), (1, 0.15), (2, 0.18), (1, 0.11), (2, 0.12), (2, 0.1), (1, 0.06), (1, 0.048), (1, 0.035), (1, 0.002)}.

	RTE		SST-2	
	acc-avg	acc-best	acc-avg	acc-best
	ElasticBERT as backbone			
JT	62.3	70.7	88.6	92.0
ALT	63.1	70.9	88.4	91.6
FasterBERT	**64.5**	72.3	**89.3**	92.1
	BERT-base as backbone			
JT	61.5	68.9	87.5	91.3
ALT	61.9	69.3	87.7	91.2
FasterBERT	**62.7**	70.5	**88.9**	92.5

Table 3: Experimental results of different PLM backbones.

Ablation studies on different PLMs

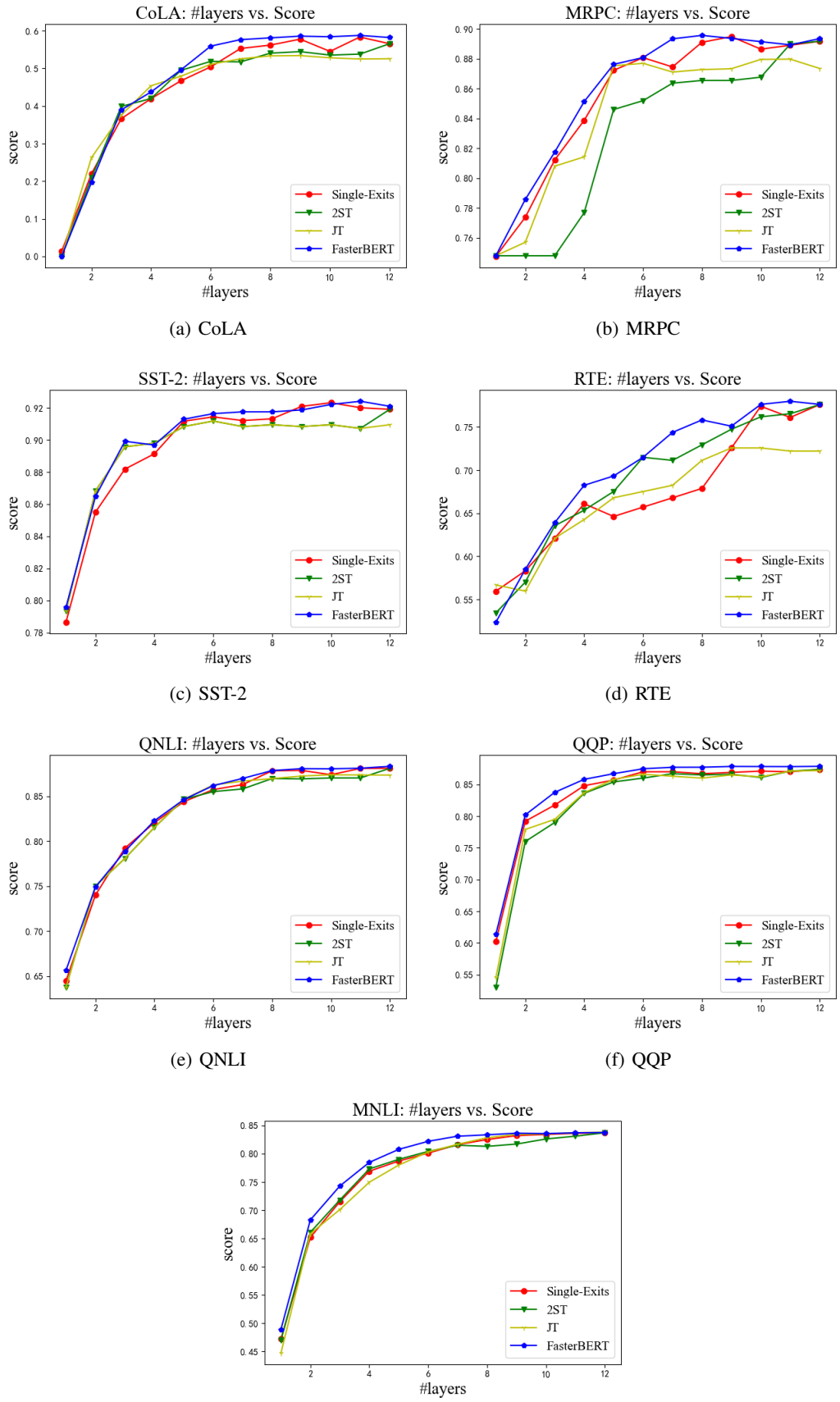
In the experiments of the main content, the PLM backbone is the ALBERT-base. In Table 3, we report the results using BERT-base and ElasticBERT as backbones. The results show that our FasterBERT works well with different pre-trained models.

References

Liu, W.; Zhou, P.; Zhao, Z.; Wang, Z.; Deng, H.; and Ju, Q. 2020. FastBERT: a Self-distilling BERT with Adaptive Inference Time. *ArXiv*, abs/2004.02178.

Xin, J.; Tang, R.; Lee, J.; Yu, Y.; and Lin, J. 2020. DeeBERT: Dynamic Early Exiting for Accelerating BERT Inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2246–2251. Online: Association for Computational Linguistics.

Xin, J.; Tang, R.; Yu, Y.; and Lin, J. 2021. BERxiT: Early Exiting for BERT with Better fine-tuning and extension to regression. In *Proceedings of the 16th conference of the European chapter of the association for computational linguistics: Main Volume*, 91–104.



(a) CoLA (b) MRPC

(c) SST-2 (d) RTE

(e) QNLI (f) QQP

(g) MNLI

Figure 2: The layer-score curves for different multi-exit BERT biaffine NER model training methods, on the ACE, CoNLL03 and Onto-4 datasets.

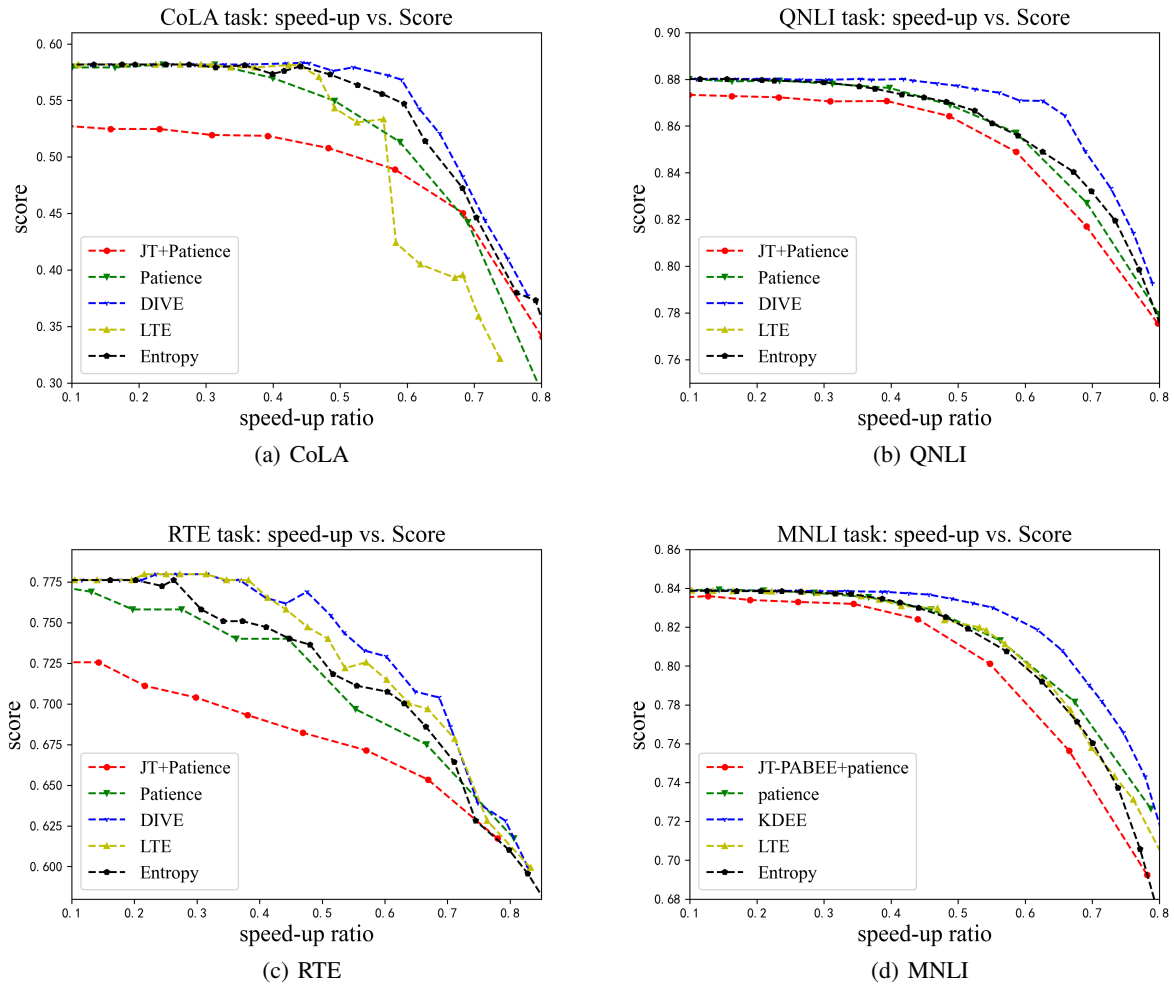


Figure 3: The speedup-score curves for different early exiting methods. The results show that our DIVE method outperforms the baseline methods by achieving better efficiency-quality trade-offs.